

Chapter 2

JavaScript and Client-Side Programming (12 hours)

1. JavaScript essentials
 - a. Data types, variables, control structures, functions
 - b. Dom manipulation, events, and form validation
 - c. Local storage and session storage
 - d. GUI interactions
 - e. JavaScript library: jQuery
2. Modern JavaScript (ES6+)
 - a. Arrow functions, destructuring, spread/rest operators
 - b. Callbacks, promises, async/await
 - c. Modules and imports
3. Client-side applications
 - a. Client-side web application development using React JS library
 - b. Traditional multi-page apps vs. single-page apps (SPAs)
 - c. Component-based UI and props
 - d. State management and data flow
 - e. Client-side routing and navigation without reloads
 - f. Fetch API: Fetching, displaying data, handling errors and loading states
 - g. Comparison of modern JavaScript libraries: React, Angular, Vue

1. JavaScript essentials

1.1 JavaScript essentials, Data types, variables, control structures, functions

Variables

Variables are the first building block you will be introduced to when learning most languages. Variables are values in your code that can represent different values each time the code runs. Here is an example of two variables in a script:

```
firstname = "Maaike";  
x = 2;
```

And they can be assigned a new value while the code is running:

```
firstname = "Edward";  
x = 7;
```

Without variables, a piece of code would do the exact same thing every single time it was run. Even though that could still be helpful in some cases, it can be made much more powerful by working with variables to allow our code to do something different every time we run it.

Declaring variables

The first time you create a variable, you declare it. And you need a special word for that: `let`, `var`, or `const`. We'll discuss the use of these three arguments shortly. The second time you call a variable, you only use the name of the existing variable to assign it a new value:

```
let firstname = "Maria";  
firstname = "Jacky";
```

`let`, `var`, and `const`

A variable definition consists of three parts: a variable-defining keyword (`let`, `var`, or `const`), a name, and a value. Some examples of variables using the different keywords:

```
let nr1 = 12;  
var nr2 = 8;  
const PI = 3.14159;
```

`let` and `var` are both used for variables that might have a new value assigned to them somewhere in the program. The difference between `let` and `var` is complex. It is related to scope.

`Var` has global scope and `let` has block scope. `var`'s global scope means that you can use the variables defined with `var` in the entire script. On the other hand, `let`'s block scope means you can only use variables defined with `let` in the specific block of code in which they were defined. Remember, a block of code will always start with `{` and end with `}`, which is how you can recognize them.

On the other hand, `const` is used for variables that only get a value assigned once—for example, the value of `pi`, which will not change. If you try reassigning a value declared with `const`, you will get an error:

```
const someConstant = 3;  
someConstant = 4;
```

This will result in the following output:

```
Uncaught TypeError: Assignment to constant variable.
```

Naming variables

When it comes to naming variables, there are some conventions in place:

- Variables start with a lowercase letter, and they should be descriptive. If something holds an age, do not call it x, but age. This way, when you read your script later, you can easily understand what you did by just reading your code.
- Variables cannot contain spaces, but they can use underscores. If you use a space, JavaScript doesn't recognize it as a single variable.

Primitive data types

JavaScript is a loosely typed language. This means that JavaScript determines the type based on the value. The type does not need to be named explicitly. For example, if you declared a value of 5, JavaScript will automatically define it as a number type. JavaScript has seven primitives: String, Number, BigInt, Boolean, Symbol, undefined, and null. We'll discuss each of them in more detail below.

1. String
2. Escape characters

There is a special character that can be used to tell JavaScript, "do not take the next character as you normally would." This is the escape character, a backslash.

In this example, the backslash can be used to ensure your interpreter doesn't see the single or double quote marks and end either string too early:

```
let str = "Hello, what's your name? Is it \"Mike\"?";
console.log(str);
let str2 = 'Hello, what\'s your name? Is it "Mike"?';
console.log(str2);
```

Output:

```
Hello, what's your name? Is it "Mike"?
Hello, what's your name? Is it "Mike"?
```

3. Number
4. BigInt
5. Boolean
6. Symbol
7. Undefined
8. null

Operators

1. Arithmetic Operators

- a. Addition (+) : Used to add numbers and strings
- b. Subtraction (-)
- c. Multiplication (*)
- d. Division (/)
- e. Exponential (**)
- f. Modulus (%)

2. Unary Operators

- a. Prefix and postfix operators (x++ and x- -)
 - b. Assignment Operators (=, +=, -=, *=, /=, %=, **=)
3. Comparison Operators (==, ===, !=, >, <, >=, <=)
4. Logical operators (and, or, not)

Arrays

Basic Array Methods

- push()
- pop()
- shift()
- unshift()
- concat()
- join()
- slice()
- splice()
- indexOf()
- lastIndexOf()
- includes()

Iteration Methods

- forEach()
- map()
- filter()
- reduce()
- reduceRight()
- find()
- findIndex()
- every()
- some()

Sorting / Reversing

- sort()
- reverse()

Conversion

- toString()
- flat()
- flatMap()

Multi Dimensional Arrays

Eg: a two-dimensional array. We can write it like this:

```
let arrOfArrays2 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
```

Objects in JS

Built-in Object Methods

- Object.keys()
- Object.values()
- Object.entries()
- Object.assign()
- Object.freeze()
- Object.seal()
- Object.create()
- Object.fromEntries()
- Object.hasOwn()
- Object.getOwnPropertyNames()

Common Operations

- Adding new properties (obj.key = value)
- Deleting properties (delete obj.key)
- Checking property (obj.hasOwnProperty())
- Looping through properties (for...in)

Logical Statements in JS

A. Conditional Statements

- a. if
- b. if ... else
- c. else if
- d. switch
- e. Conditional ternary operator (expression ? statement for true : statement associated with false;)

B. Loops

- a. for loop
- b. while loop
- c. do...while loop
- d. for...of (arrays) : used to access array elements
- e. for...in (objects): used to access keys of object

C. Jump Statements

- a. break
- b. continue

Functions in JavaScript

- A. Function Declaration
- B. Types of Functions
 - a. Functions with no parameter and no return type
 - b. Functions with no parameter but have return type
 - c. Functions with parameter but no return type
 - d. Functions with both parameter and return type
- C. Arrow Functions (ES6+)
- D. Anonymous Function (No name, used in callbacks.)

```
setTimeout(function() {  
    console.log("Hello after 2 seconds");  
}, 2000);
```
- E. Default Parameters
- F. Nested Functions

DOM

1. Introduction to DOM

DOM stands for Document Object Model. It represents an HTML document as a tree structure so that JavaScript can access and modify HTML elements dynamically. Every HTML element becomes an object in the DOM.

Example HTML structure:

```
<html>  
  <body>  
    <h1>Hello World</h1>  
  </body>  
</html>
```

2. Selecting DOM Elements

a) getElementById()

Used to select an element by its unique ID.

Syntax:

```
const element = document.getElementById("id");
```

Example:

```
const para = document.getElementById("para");
```

b) getElementsByClassName()

Selects elements by class name.

Syntax:

```
const elements = document.getElementsByClassName("className");
```

Example:

```
const items = document.getElementsByClassName("text");
```

c) `getElementsByName()`

Selects elements by tag name.

Syntax:

```
const elements = document.getElementsByName("tag");
```

Example:

```
const paras = document.getElementsByName("p");
```

d) `querySelector()`

Selects the first matching element using CSS selectors.

Syntax:

```
const element = document.querySelector("selector");
```

Example:

```
const first = document.querySelector(".text");
```

e) `querySelectorAll()`

Selects all matching elements.

Syntax:

```
const elements = document.querySelectorAll("selector");
```

Example:

```
const all = document.querySelectorAll(".text");
```

3. Changing Content in DOM

`innerText`

Changes visible text.

Syntax:

```
element.innerText = "New Text";
```

`innerHTML`

Changes HTML content.

Syntax:

```
element.innerHTML = "<b>Bold Text</b>";
```

`textContent`

Changes all text content.

Syntax:

```
element.textContent = "Plain Text";
```

4. Styling Elements Using JavaScript

Inline CSS Styling

Example:

```
const box = document.getElementById("box");
box.style.backgroundColor = "blue";
box.style.color = "white";
box.style.padding = "20px";
```

Managing Classes

Syntax:

```
box.classList.add("active");
box.classList.remove("active");
box.classList.toggle("active");
```

CSS Example:

```
.active {
  background-color: green;
  color: white;
}
```

5. DOM Events

Using `addEventListener()`

HTML:

```
<button id="btn">Click Me</button>
```

JavaScript:

```
const btn = document.getElementById("btn");

btn.addEventListener("click", function() {
  alert("Button clicked");
});
```

Mouse Events

Example:

```
btn.addEventListener("mouseover", () => {
  console.log("Mouse Over Button");
});
```

Keyboard Events

HTML:

```
<input id="inputBox">
```

JavaScript:

```
const inputBox = document.getElementById("inputBox");

inputBox.addEventListener("keyup", () => {
```

```
    console.log(inputBox.value);  
  });
```

6. Creating and Removing Elements

Create Element

```
const div = document.createElement("div");  
div.innerText = "This is a new div";  
  
document.body.appendChild(div);
```

Remove Element

```
div.remove();
```

7. Mini Project – Dynamic ToDo List

HTML:

```
<input id="task" placeholder="Enter task">  
<button id="addBtn">Add Task</button>  
<ul id="taskList">  
  <li></li>  
</ul>
```

JavaScript:

```
const addBtn = document.getElementById("addBtn");  
const input = document.getElementById("task");  
const list = document.getElementById("taskList");  
  
addBtn.addEventListener("click", (e) => {  
  e.preventDefault();  
  const task = input.value;  
  if (task === "") {  
    alert("Please enter a task");  
    return;  
  }  
  const li = document.createElement("li");  
  li.innerText = task;  
  
  list.appendChild(li);  
  input.value = "";  
});
```

Local Storage (Web Storage API)

Local Storage stores data in the browser permanently (until you manually clear it).

Key Features:

- Data remains even after closing the browser
- Storage limit: ~5–10MB
- Stores data as key–value pairs (strings only)

```
// Store data
localStorage.setItem("username", "Sunil");

// Get data
let user = localStorage.getItem("username"); console.log(user);

// Remove specific item localStorage.removeItem("username");

// Clear all data
localStorage.clear();
```

Session Storage

Session Storage is similar to local storage, but: Data is cleared when the tab or browser is closed. Session Storage Session Storage is similar to local storage, but: Data is cleared when the tab or browser is closed

```
// Store data
sessionStorage.setItem("sessionUser", "Sunil");
// Get data
let user = sessionStorage.getItem("sessionUser");
console.log(user);
// Remove specific item
sessionStorage.removeItem("sessionUser");
// Clear all
sessionStorage.clear();
```

Form Validation in JavaScript

Form validation ensures users enter correct and complete data before submitting a form.

HTML:

```
<form id="myForm" onsubmit="return validateForm()">
  <label>Name:</label>
  <input type="text" id="name" />
  <span id="nameError" style="color:red;"></span>
  <br><br>
  <label>Email:</label>
  <input type="text" id="email" />
  <span id="emailError" style="color:red;"></span>
  <br><br>
  <label>Password:</label>
```

```

"></span>
<br><br>
<button type="submit">Submit</button>
</form>

```

JS:

```

function validateForm() {
    let name = document.getElementById("name").value.trim();
    let email = document.getElementById("email").value.trim();
    let password =
document.getElementById("password").value.trim();
    let isValid = true;

    // Name validation
    if (name === "") {
        document.getElementById("nameError").innerText = "Name is
required";
        isValid = false;
    } else {
        document.getElementById("nameError").innerText = "";
    }

    // Email validation using regex
    let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (email === "") {
        document.getElementById("emailError").innerText = "Email is
required";
        isValid = false;
    } else if (!emailPattern.test(email)) {
        document.getElementById("emailError").innerText = "Invalid
email format";
        isValid = false;
    } else {
        document.getElementById("emailError").innerText = "";
    }

    // Password validation
    if (password === "") {
        document.getElementById("passError").innerText = "Password
is required";
        isValid = false;
    } else if (password.length < 6) {
        document.getElementById("passError").innerText = "Password
must be at least 6 characters";
        isValid = false;
    }
}

```

```
    } else {  
        document.getElementById("passError").innerText = "";  
    }  
  
    return isValid; // prevents form submit if false  
}
```

jQuery

jQuery is arguably the most famous JavaScript library. It was great to use in the past, when it would be compiled into the latest version of JavaScript for the specific browser. Nowadays, it is just a different way of writing some of the things we have seen in the book. You can recognize jQuery easily by the amount of dollar signs in the code. You can also tell if a website is using jQuery if you type \$ or jQuery into the console of the website, and it returns the jQuery object. The jQuery library is mainly focused on selecting HTML elements from the DOM and interacting with and manipulating them. It roughly looks like this:
\$(selector).action();

jQuery is a fast, lightweight JavaScript library that simplifies common tasks like DOM manipulation, event handling, animations, and AJAX using easy-to-write syntax.

```
<html>
<head>
  <script
    src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/
    jquery.min.js"></script>
</head>
<body>
  <p>Let's play a game!</p>
  <p>Of hide and seek...</p>
  <p class="easy">I'm easy to find!</p>
  <button id="hidebutton">Great idea</button>
  <button id="revealbutton">Found you!</button>
  <script>
    $(document).ready(function () {
      $("#hidebutton").click(function () {
        $("p").hide();
      });
      $("#revealbutton").click(function () {
        $(".easy").show();
      });
    });
  </script>
</body>
</html>
```

1. Why jQuery?

Before jQuery, JavaScript code was:

- Long and repetitive
- Browser-compatibility heavy

jQuery follows the idea:

“Write less, do more.”

2. How to Use jQuery

Include jQuery (CDN)

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

3. Basic Syntax

```
$(selector).action();
```

Example

```
$("p").hide();
```

//Hides all <p> elements

4. Document Ready

Ensures HTML is fully loaded before running JS:

```
$(document).ready(function () {  
    console.log("Page loaded");  
});
```

Shortcut:

```
$(function () {  
    console.log("Ready");  
});
```

5. Selectors

Selector	Meaning
\$("p")	All <p>
\$("#id")	Element with id
\$(".class")	Elements with class
\$("#div p")	<p> inside <div>

`$("#hero h1")`

6. Events

```
$("#btn").click(function () {  
    alert("Button clicked");  
});
```

Common events:

- `click()`
- `dblclick()`
- `mouseenter()`
- `mouseleave()`
- `keyup()`
- `change()`

7. DOM Manipulation

Change Text / HTML

```
$("#title").text("Hello");  
$("#box").html("<b>Bold</b>");
```

CSS

```
$("#p").css("color", "red");
```

Add / Remove Classes

```
$("#div").addClass("active");  
$("#div").removeClass("active");  
$("#div").toggleClass("active");
```

8. Effects & Animations

```
$("#box").hide();  
$("#box").show();  
$("#box").toggle();  
$("#box").fadeIn();  
$("#box").fadeOut();  
$("#box").slideUp();  
$("#box").slideDown();
```

9. AJAX (Very Important)

AJAX stands for Asynchronous JavaScript and XML, which is a misnomer, because nowadays it is more common to use JSON instead of XML. We use it to make calls from the frontend to the backend, without refreshing the page (asynchronously). AJAX is not a programming language or a library, it is a combination of the built-in XMLHttpRequest object in the browser and the JavaScript language.

```
$.ajax({
  url: "data.php",
  method: "GET",
  success: function (response) {
    console.log(response);
    // jquery or DOM
  }
});
```

Shortcut:

```
$.get("data.php", function (data) {
  console.log(data);
});
```

10. Form Handling Example

```
$("#form").submit(function (e) {
  e.preventDefault();
  let name = $("#name").val();
  alert(name);
});
```

11. jQuery vs JavaScript

jQuery	JavaScript
Short code	Longer
Easy	More control
Older projects	Modern JS preferred

2.2 Modern JavaScript (ES6+)

Arrow Functions

Arrow functions are a shorter syntax for writing functions introduced in ES6. They provide a cleaner and more readable way to write functions.

Syntax:

```
// Normal function
function add(a, b) {
    return a + b;
}

// Arrow function
const add = (a, b) => {
    return a + b;
}

// Short form
const add = (a, b) => a + b;
```

Destructuring

Destructuring allows extracting values from arrays or objects into variables easily.

Array Destructuring:

```
const numbers = [10, 20, 30];
const [a, b, c] = numbers; // [10, 20, 30]
console.log(a); // 10
```

Skipping Values

```
const [first, , third] = numbers;
```

```
//first = 10
```

Object Destructuring

```
const user = {
    name: "Ram",
    age: 22,
    city: "Kathmandu",
    //country:
};

const { name, age } = user;
```

Renaming Variables

```
const { name: fullName } = user;  
//fullName: "Ram"
```

Default Values

```
const { country = "Nepal" } = user;
```

Spread and Rest Operators (...)

3.1 Spread Operator

The spread operator is a special operator. It consists of three dots used before a referenced expression or string, and it spreads out the arguments or elements of an array. Used to **expand elements** of arrays or objects.

Array Example

```
const arr1 = [1, 2];  
const arr2 = [...arr1, 3, 4];
```

Other Example:

```
let spread = ["so", "much", "fun"];  
let message = ["JavaScript", "is", ...spread, "and", "very",  
"Powerful"];  
  
//The value of this array becomes: ['JavaScript', 'is', 'so',  
'much', 'fun', 'and', 'very', 'powerful']
```

Object Example

```
const user = { name: "Sita" };  
const newUser = { ...user, age: 20 };
```

Copying Arrays/Objects

```
const copy = [...arr1];
```

3.2 Rest Operator

Similar to the spread operator, we have the rest parameter. It has the same symbol as the spread operator, but it is used inside the function parameter list. Used to **collect remaining elements** into an array.

```
function sum(...numbers) {  
    return numbers.reduce((a, b) => a + b, 0);  
}
```

Rest with Destructuring

```
const [first, ...rest] = [1, 2, 3, 4];  
//rest=[2,3,4]
```

Other Example:

```
function someFunction(param1, ...param2) {  
    console.log(param1, param2);  
}  
  
someFunction("hi", "there!", "How are you?");  
// Output: hi [ 'there!', 'How are you?' ]
```

Synchronous and Asynchronous Programming in JS

JavaScript is:

- Single-threaded
- Has one Call Stack
- Executes code using an Execution Context

Synchronous JavaScript executes code sequentially, where each operation blocks the next one until it finishes. The next statement waits for the current statement to complete.

Eg:

```
console.log("A");

function add(x, y) {
  return x + y;
}
console.log(add(2, 3));
console.log("B");
```

Output:

```
A
5
B
```

Asynchronous JavaScript allows long-running tasks to execute without blocking the main thread. JS starts the task, moves on, and handles the result later.

Eg:

```
console.log("Start");
setTimeout(() => {
  console.log("Timeout");
}, 2000);
console.log("End");
```

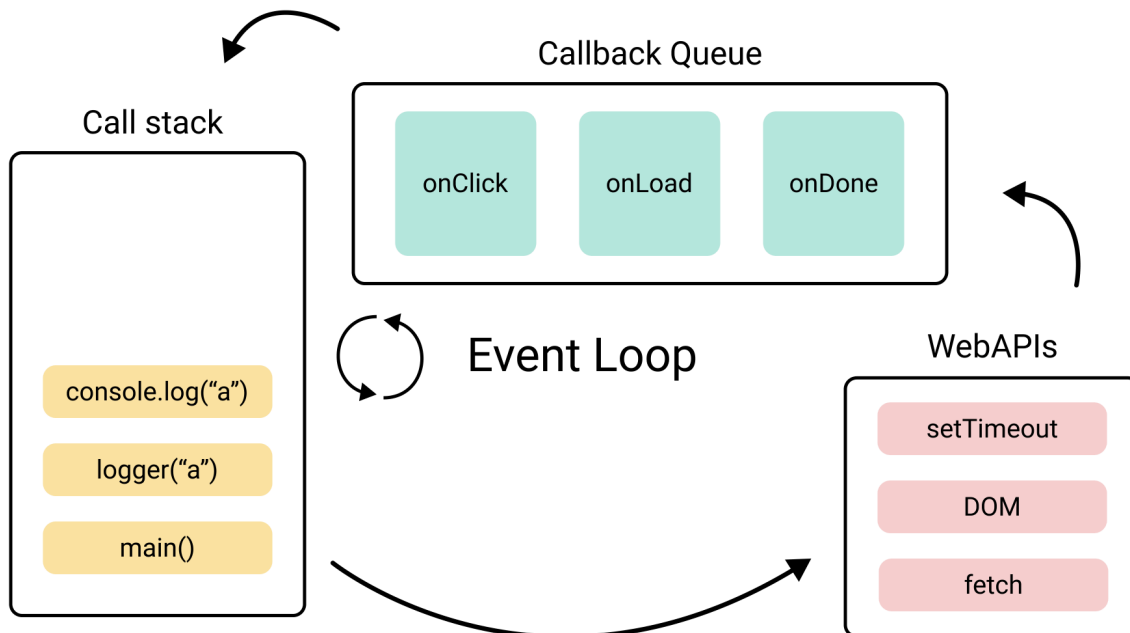
Output

```
Start
End
Timeout
```

How It Actually Works (Internals)

1. `setTimeout()` → sent to Web API
2. Timer runs independently
3. After 2s → callback goes to Callback Queue
4. Event Loop checks:
 - Call stack empty?
 - If yes → push callback to stack
5. Callback executes

Event Loop (Heart of Async JS)



Event Loop Rule

Microtask Queue executes before Callback Queue

Eg:

```
console.log("Start");  
  
setTimeout(() => {  
    console.log("Timeout");  
}, 0);  
  
Promise.resolve().then(() => {  
    console.log("Promise");  
});  
  
console.log("End");
```

Output:

Start End Promise Timeout

Why?

- Promise → Microtask Queue
- setTimeout → Callback Queue
- Microtasks always execute first

Callback Queue (Macrotask Queue)

Also called Task Queue

Examples:

- setTimeout
- setInterval
- setImmediate (Node)
- DOM events
- AJAX callbacks (XHR)

Microtask Queue (Higher Priority)

Examples:

- Promise.then / catch / finally
- queueMicrotask()
- MutationObserver
- process.nextTick() (Node – even higher)

* Microtasks ALWAYS execute before callback queue

Asynchronous Techniques

A. Callback

A callback is a function passed as an argument to another function and executed later. It is just a function that takes another function as an argument, which is then called when the rest of the initial function has finished. In other words, it's just a function calling a function, like this:

```
function doSomething(callback) {  
    callback();  
}  
  
function sayHi() {  
    console.log("Hi!");  
}  
  
doSomething(sayHi);
```

The `doSomething()` function, which is created with the parameter `callback`, is just calling whatever function is being passed in as an argument. We call it using the `sayHi()` function as an argument, so this code snippet is just a very complicated way to get `Hi!` printed to the console.

Some JavaScript built-in functions work with this callback principle, for example, the `setTimeout()` and `setInterval()` functions. They will take a function that is executed after a certain time in the case of a timeout and every certain amount of time for the specified interval. We have seen these already, but just as a reminder:

```
setInterval(encourage, 500);

function encourage() {
    console.log("You're doing great, keep going!");
}
```

The functions that are inserted as arguments are called `callbacks` here.

B. Promises

A Promise is a special object that connects code that needs to produce a result and the code that needs to use this result in the next step. A Promise is an object that represents the eventual completion or failure of an asynchronous operation and its resulting value. Promises help avoid `callback hell` and make asynchronous code more readable and manageable.

When we create a Promise, we give it a function. In the following example, we use a convention that we have seen a lot; we are creating a function on the spot. So, inside the argument list we are defining the function, often done using arrow functions as well. This function needs two parameters, and these parameters are `callbacks`. We have called them `resolve` and `reject` here.

When `resolve()` is called, the Promise is presumed to be successful and whatever is between the arrows is returned and used as input for the `then` method on the Promise object. If `reject()` is called, the Promise failed and the `catch()` method on the Promise object (if present) is executed with the argument of the `reject()` function.

A promise can be in one of three states:

1. Pending – Initial state, neither fulfilled nor rejected
2. Fulfilled – Operation completed successfully
3. Rejected – Operation failed

Once a promise is fulfilled or rejected, it is settled and cannot change its state.

1.3 Creating a Promise

```
const myPromise = new Promise((resolve, reject) => {
```

```

    let success = true;
    if (success) {
        resolve("Operation successful");
    } else {
        reject("Operation failed");
    }
});

```

1.4 Consuming a Promise

```

myPromise
    .then(result => {
        console.log(result);
    })
    .catch(error => {
        console.log(error);
    })
    .finally(() => {
        console.log("Promise settled");
    });

```

- `.then()` → runs when promise is fulfilled
- `.catch()` → runs when promise is rejected
- `.finally()` → runs in both cases

Example:

```

let promise = new Promise(function (resolve, reject) {
    // do something that might take a while
    // let's just set x instead for this example
    let x = 20;
    if (x > 10) {
        resolve(x); // on success
    } else {
        reject("Too low"); // on error
    }
});

promise.then(
    function (value) {
        console.log("Success:", value);
    },
    function (error) {
        console.log("Error:", error);
    }
)

```

```
);
```

1.5 Promise Chaining

```
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Each `.then()` returns a new promise.

1.6 Promise.all(), Promise.race(), Promise.any()

```
Promise.all([p1, p2, p3]); // Resolves when all promises resolve
Promise.race([p1, p2]); // Resolves/rejects with first settled
promise
Promise.any([p1, p2]); // Resolves with first fulfilled promise
```

C. Async/ Await

We have just seen the Promise syntax. With the `async` keyword, we can make a function return a Promise. This makes the Promises nicer to read and look a lot like synchronous (non-concurrent) code. We can use this Promise just like we learned in the previous section, or we can use the more powerful `await` keyword to wait until the Promise is done. `await` only works in an asynchronous function.

`Async` and `await` are syntactic sugar over promises, making asynchronous code look synchronous.

```
async function getData() {
  return "Hello";
}
//Always returns a promise
```

2.3 await Keyword

```
async function fetchData() {
  const response = await fetch(url);
  const data = await response.json();
  console.log(data);
}
```

- `await` pauses execution until the promise resolves
- Can only be used inside `async` functions

Example:

```
function saySomething(x) {
```

```

        return new Promise(resolve => {
            setTimeout(() => {
                resolve("something" + x);
            }, 2000);
        });
    }
    async function talk(x) {
        const words = await saySomething(x);
        console.log(words);
    }
    talk(2);
    talk(4);
    talk(8);

```

We call the asynchronous function `talk()` three times in a row with no break. Each of these function calls is awaiting the `saySomething()` function. The `saySomething()` function contains a new Promise that is being resolved with a `setTimeout()` function that waits two seconds before resolving with the value of something plus x. So after two seconds, the three functions are done at the same time

2.4 Error Handling with try/catch

```

async function loadData() {
    try {
        const res = await fetch(url);
        const data = await res.json();
        console.log(data);
    } catch (error) {
        console.error(error);
    }
}

```

2.5 Parallel Execution with await

```

const p1 = fetch(url1);
const p2 = fetch(url2);

const res1 = await p1;
const res2 = await p2;

```

Modules and Imports

3.1 What are Modules?

Modules allow you to split code into separate files and reuse them. Each module has its own scope.

3.2 Exporting from a Module

Named Export

```
export const PI = 3.14;
export function add(a, b) {
    return a + b;
}
```

Default Export

```
export default function greet() {
    console.log("Hello");
}
```

3.3 Importing a Module

Named Import

```
import { PI, add } from './math.js';
```

Default Import

```
import greet from './greet.js';
```

Import Everything

```
import * as math from './math.js';
```

3.4 Module Usage in HTML

```
<script type="module" src="app.js"></script>
```

Client Side Application (React JS)

A client-side web application:

- Runs mainly in the browser
- Uses JavaScript to update UI dynamically
- Communicates with the server via APIs (JSON)
- Does not reload pages for every user action

Role of React

React is a JavaScript library used to:

- Build fast, interactive user interfaces
- Break UI into reusable components
- Efficiently update the DOM using Virtual DOM

“React does not replace HTML/CSS/JS — it organizes them better.”

Traditional Multi-Page Apps vs Single-Page Apps (SPA)

Multi-Page Application

A **Multi-Page Application** loads a **new HTML page from the server** every time the user clicks a link or submits a form.

How it works

- Browser sends a request to the server
- Server processes it and returns a new HTML page
- The browser reloads the entire page

Examples

- Old PHP websites
- JSP / ASP.NET websites
- Traditional e-commerce sites (older versions)

Characteristics

- Each page has its own URL
- Full page reload on navigation
- Server handles most of the logic
- Slower user experience due to reloads

Advantages

- Simple architecture

- SEO-friendly by default
- Easy to build for small websites

Disadvantages

- Poor user experience (frequent reloads)
- More bandwidth usage
- Slower performance

Single-Page Applications (SPA)

A Single-Page Application loads only one HTML page once, and updates content dynamically using JavaScript without reloading the page.

How it works

- One HTML file is loaded initially
- JavaScript (React) updates UI dynamically
- Data is fetched using APIs (Fetch / Axios)
Routing handled on the client side

Examples

- Gmail
- Facebook
- Google Drive
- React, Angular, Vue apps

Characteristics

- No full page reload
- Fast and smooth navigation
- Uses client-side routing
- Communicates with backend via APIs

Advantages

- Better user experience
- Faster interactions
- Reduced server load after initial load

Disadvantages

- Initial load can be slow
- SEO needs extra handling
- Requires JavaScript enabled

MPA reloads the entire page on each request, whereas SPA loads a single page once and updates content dynamically without reloading.

Component-Based UI and Props

A component is a reusable, independent piece of UI in a React application. Each component controls its own structure and behavior. React applications are built by combining multiple small components to form a complete UI.

A web page can be divided into:

- Header
- Sidebar
- Content
- Footer

*Each part is a **separate component**.*

Types of Components

1. **Functional Components** (most common)
2. **Class Components** (older approach)

*Modern React uses **functional components + hooks***

Eg:

```
function Welcome() {  
    return props  
}
```

Props (Properties)

Props are **inputs passed from a parent component to a child component**. They make components **dynamic and reusable**. Props are **read-only** (cannot be changed by the child).

* Props flows in one way only in React i.e from parent to child only.

Component-based UI divides the application into reusable components, and props are read-only data passed from parent to child components.

Props Using Typescript

```
type StudentProps = {  
  name: string;  
  age: number;  
};
```

Usage:

```
const Student = ({ name, age }: StudentProps) => {  
  return (  
    <p>  
      Name: {name}, Age: {age}  
    </p>  
  );  
};
```

Pass Props form Parent:

```
const App = () => {  
  return <Student name="Sunil" age={21} />;  
};  
  
export default App;
```

Function as Props

Component accepting function

```
type CounterProps = {  
  increment: () => void;  
};
```

```
const CounterButton = ({ increment }: CounterProps) => {  
  return <button onClick={increment}>Increase</button>;  
};
```

Parent:

```
const App = () => {  
  const handleIncrement = () => {  
    console.log("Incremented");  
  };  
  return <CounterButton increment={handleIncrement} />;  
};
```

Props with Children

Component:

```
type CardProps = {  
  children: React.ReactNode;  
};  
  
const Card = ({ children }: CardProps) => {  
  return <div className="card">{children}</div>;  
};
```

Parent:

```
<Card>  
  <h2>Hello Students</h2>  
</Card>
```

State Management in React

State is an object that holds data that can change over time in a component. Changing the state triggers a re-render of the component. Think of state as the “memory” of a component for dynamic behavior.

Eg:

```
import React, { useState } from "react";

const Counter = () => {
  const [count, setCount] = useState(0); // initial state is 0
  const increment = () => {
    setCount(count + 1); // updates state and re-renders
    component
  };
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increase</button>
    </div>
  );
};

export default Counter;
```

Here, `useState` returns current value + function to update it. Calling `setCount` updates the state and re-renders the UI.

Rules of State

1. State should be local to the component (unless using global state management).
2. Do not modify state directly; always use the setter function.
3. State updates are **asynchronous**.

Example of Dynamic Input from a form

```
const Parent = () => {
  const [name, setName] = useState("");
  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={ (e) => setName(e.target.value) }
      />
      <Child name={name} />
    </div>
  );
};

const Child = ({ name }: { name: string }) => <p>Hello,
{name}!</p>;
```

State Management in Larger Apps

- For complex apps, use global state libraries:
 - Redux
 - Zustand
 - Context API
- Helps share state across multiple components without prop-drilling.

Client-Side Routing (Without Page Reload)

Routing determines which view or component should be displayed based on the URL. Traditional websites: every route triggers a full page reload (Multi-Page Apps).

In Single-Page Applications (SPA), routing is handled on the client using JavaScript. Only the relevant component changes, page is not reloaded. Faster navigation → smoother user experience.

How It Works in React

- Uses libraries like React Router.
- URL changes are captured by JavaScript.
- Corresponding React component is rendered dynamically.

Example of Routes:

```
import { BrowserRouter as Router, Routes, Route, Link } from
"react-router-dom";

const Home = () => <h2>Home Page</h2>;

const About = () => <h2>About Page</h2>;

const Contact = () => <h2>Contact Page</h2>;

const App = () => {

  return (

    <Router>

      <nav>

        <Link to="/">Home</Link> |

        <Link to="/about">About</Link> |

        <Link to="/contact">Contact</Link>

      </nav>

      <Routes>

        <Route path="/" element={<Home />} />

        <Route path="/about" element={<About />} />

        <Route path="/contact" element={<Contact />} />

      </Routes>

    </Router>

  );

};

export default App;
```

Routing in Next.js

Next.js has a **file-based routing system**. Each file in the pages directory automatically becomes a **route**.

Example:

```
pages/  
index.tsx    → "/"  
about.tsx    → "/about"  
contact.tsx  → "/contact"  
blog/[id].tsx → Dynamic blog route
```

Fetch API – Data Fetching in React

React apps often:

- Consume REST APIs
- Display dynamic data
- Work with backend servers

Eg:

```
fetch(url)  
  .then(res => res.json())  
  .then(data => console.log(data))
```

Common Fetching Pattern in React

- Use useEffect
- Fetch once on component load
- Update state with data

Key concepts to emphasize:

- Asynchronous behavior
- Error handling (try/catch)
- Conditional rendering

Comparison of Modern JS Libraries

React

- Library (not full framework)
- Uses **JSX**
- Flexible & lightweight
- Huge ecosystem

Angular

- Full-fledged framework
- Uses TypeScript
- Built-in tools (routing, forms)
- Steeper learning curve

Vue

- Progressive framework
- Easy to learn
- Template-based syntax
- Popular in Asia